WebhookGuard — Simple Explanation Guide

Automated Webhook API Testing & Reliability Platform | Complete Exam & Interview Preparation

IF YOU ONLY REMEMBER 10 THINGS

If you only have 5 minutes before your interview or viva, remember these 10 core points:

- **1. What WebhookGuard Is** — It is an automated testing platform for webhook APIs. It acts like an automated quality inspector that sends real test webhooks (valid, broken, tampered, slow) to an API to see if it responds safely or crashes.
- **2. The Core Problem It Solves** — In modern apps, services like Stripe, GitHub, or Shopify notify your backend via webhooks. If your webhook API crashes or accepts fake notifications, you lose money or get hacked. WebhookGuard catches these bugs before production.
- **3. Real Network Execution (Not Fake)** — WebhookGuard does NOT simulate fake test results. AWS Lambda makes genuine outbound HTTP/HTTPS requests over the public internet to the target webhook URL and measures real milliseconds latency.
- **4. Negative & Security Testing** — Good software testing doesn't just test happy paths. WebhookGuard intentionally sends malformed JSON, missing fields, and tampered cryptographic HMAC signatures. If your API accepts a fake signature with HTTP 200, WebhookGuard flags a 'SECURITY FAILURE'.
- **5. 100% Serverless on AWS** — The entire application runs in the cloud on AWS (ap-south-2 Hyderabad). There are zero traditional EC2 virtual machines running. You pay only when a request runs, and it scales to zero when idle.
- **6. Frontend on AWS S3 Static Website** — The user interface is built in React + TypeScript + Tailwind CSS and hosted on an Amazon S3 website bucket. Anyone with the URL can open it from any browser worldwide.
- **7. Backend on AWS Lambda + Express** — The server code runs inside AWS Lambda using Node.js 22. It starts up only when an API call arrives, runs for a few milliseconds or seconds, and turns off.
- **8. API Gateway as the Door** — Amazon API Gateway (HTTP API v2) acts as the secure front door. It receives HTTP requests from the browser, handles CORS, and forwards them to AWS Lambda.
- **9. Database in Amazon DynamoDB** — All webhook endpoints, test suites, test runs, and test results are stored in 4 Amazon DynamoDB NoSQL tables. Your data stays safe and persists even after refreshing the page or restarting your laptop.
- **10. CI/CD Quality Gate Endpoint** — WebhookGuard has a special endpoint (POST /api/test-runs?failOnErrors=true). If any test fails, it returns HTTP 422, allowing automated tools like GitHub Actions to automatically stop a bad software release.

1. PROJECT IN ONE MINUTE (SPOKEN ELEVATOR PITCH)

What you can say out loud in 30 to 60 seconds when someone asks: 'Tell me about your project.'

"WebhookGuard is an automated testing platform for webhook APIs. In modern web applications, external services like Stripe for payments or GitHub for code changes send background notifications called webhooks to our servers. If our server cannot handle malformed payloads, fake signatures, or sudden timeouts, the business suffers outages, data corruption, or security breaches."

"WebhookGuard solves this by giving developers an intuitive web dashboard where they can register their webhook endpoints, choose pre-built test suites, and click 'Run Test'. Our AWS Lambda backend immediately sends real HTTP requests over the internet to test positive cases, broken JSON, missing fields, timeout tolerance, retry recovery, and invalid HMAC signatures."

"The results and response latencies are analyzed in real time and saved to Amazon DynamoDB. The entire project is live in the cloud on AWS using S3, API Gateway, Lambda, and DynamoDB, meaning it works 24/7 from any browser without needing my personal laptop running."

2. SIMPLE REAL-WORLD EXAMPLE: THE PAYMENT WEBHOOK

A concrete story to help anyone understand webhooks within 20 seconds.

Imagine an online shopping store like Amazon or Shopify:

1. A customer buys a pair of shoes and enters credit card details on Stripe's payment page.

2. Stripe charges the card successfully. Now Stripe needs to notify the shoe store's backend: 'Payment of $50 succeeded for Order #123.'

3. Stripe sends an automated HTTP POST request to the shoe store's webhook URL: https://shoestore.com/api/webhooks/payment.

4. The shoe store's API receives this message, checks the cryptographic signature, marks the order as paid, and tells the warehouse to ship the shoes.

- What could go wrong? The shoe store's developer might have a bug. What if Stripe sends unexpected unicode characters? Does the server crash? What if an attacker sends a fake message claiming 'Order #999 was paid' with a forged signature? Does the store ship free shoes? What if the database is busy and takes 4 seconds to reply? Does Stripe timeout and cancel?
- **How WebhookGuard tests it:** WebhookGuard pretends to be Stripe, an attacker, and a slow network all in one. It sends genuine requests, malformed requests, and tampered signatures to the shoe store's webhook.
- **What result WebhookGuard gives:** A clear report showing PASS, FAIL, or SECURITY FAILURE, with millisecond response times and exact payload diffs so developers can fix bugs before launching to production.

3. WHAT IS A WEBHOOK?

Understanding the difference between traditional API polling and webhooks.

A webhook is an automated, event-driven HTTP request sent from one system to another when something specific happens.

Think of it like a phone notification versus constantly checking your mailbox: Traditional APIs require you to ask the server every 5 seconds: 'Is the package here yet?' (called polling). A webhook is when the delivery driver rings your doorbell the exact second the package arrives.

- **Why is it used?:** It saves server resources, bandwidth, and ensures real-time updates without polling.
- **Who sends the request?:** The service where the event occurred (e.g. Stripe, GitHub, Shopify, Slack).
- **Who receives it?:** Your backend application (a specific URL endpoint listening for incoming data).
- **What is a payload?:** The payload is the actual data package carried inside the request, almost always formatted as JSON (e.g. { "event": "payment.success", "amount": 4999 }).

4. WHAT IS AN API AND HTTP BASICS?

The foundational communication building blocks explained simply.

API stands for Application Programming Interface. It is a set of rules and contracts that allows two different computer programs to talk to each other over a network.

An HTTP Request is a message sent by a client (like a browser or WebhookGuard) asking a server to do something. An HTTP Response is the message the server sends back with the result.

- **GET:** Asks the server to retrieve data (e.g. get list of endpoints). GET never modifies data.
- **POST:** Submits new data to the server (e.g. create an endpoint, run a test suite, trigger a webhook).
- **PUT:** Replaces or completely updates existing data.
- **DELETE:** Removes data from the server.
- **Headers:** Key-value metadata sent alongside the request (e.g. Content-Type: application/json, X-Signature: sha256=abc...).
- **JSON (JavaScript Object Notation):** A simple, human-readable text format for structured data made of keys and values (e.g. {"status": "ok", "count": 5}).

5. WHAT DOES WEBHOOKGUARD ACTUALLY DO? (END-TO-END FLOW)

Follow a test run from the user's mouse click down to the database and back.

Here is the exact step-by-step path taken every time a user runs a test in WebhookGuard:

- **Step 1 (User Action):** The user opens the WebhookGuard website, selects a Test Suite (e.g. Core Payment Reliability Suite), and clicks 'Run Test Suite'.
- **Step 2 (Frontend Dispatch):** The React frontend sends an HTTP POST request to Amazon API Gateway (/api/test-runs) with the endpoint ID and suite ID.
- **Step 3 (API Gateway Routing):** API Gateway inspects the request, validates CORS headers, and invokes the AWS Lambda backend function.
- Step 4 (Lambda Test Runner Execution): The Node.js Lambda function retrieves the test scenarios from DynamoDB and executes them in sequence using real outbound fetch() requests across the public internet.
- **Step 5 (External Webhook Target):** The target webhook receiver (e.g. a payment receiver or mock endpoint) receives the HTTP request and returns an HTTP response code (e.g. 200, 400, 401).
- **Step 6 (Analysis Engine):** WebhookGuard measures response time in milliseconds, checks if the actual status matches the expected status, and checks if an unauthenticated request was mistakenly accepted.
- **Step 7 (DynamoDB Storage):** The test run summary and all granular test results are saved into Amazon DynamoDB tables (WebhookGuard-TestRuns and WebhookGuard-TestResults).
- **Step 8 (Frontend Display):** The frontend receives the JSON response, updates the UI progress bar, and displays detailed PASS, FAIL, or SECURITY FAILURE badges.

6. FRONTEND VS BACKEND: ROLES & ANALOGY

The restaurant analogy that anyone can visualize instantly.

Think of WebhookGuard like a fine-dining restaurant:

- The Frontend is the dining room, tables, menu, and waiter. It looks beautiful, takes your order, and presents the food nicely.

- The Backend is the kitchen, chefs, and pantry behind the swinging doors. It takes the order, handles the fire, cooks the meal, and stores ingredients in the refrigerator.

- Communication: The waiter (HTTP API calls) carries orders between the dining room and the kitchen.

- **Frontend in WebhookGuard:** Built with React 18, TypeScript, Tailwind CSS, and Vite. It renders the dashboard, interactive charts, forms for adding endpoints, and live test run execution viewers.
- **Backend in WebhookGuard:** Built with Node.js 22 and Express. It houses the TestRunnerService, calculates HMAC cryptographic signatures, enforces timeouts via AbortController, and communicates with DynamoDB.
- **How they talk:** The frontend sends standard HTTP JSON requests (GET, POST, PUT, DELETE) over HTTPS to the backend's API Gateway URL.

7. AWS EXPLANATION: THE CLOUD SERVICES WE USE

What AWS is, and a plain-English breakdown of every service running in our deployment.

AWS (Amazon Web Services) is the world's largest cloud computing platform. Instead of buying physical computer towers, installing operating systems, and keeping them running in our bedroom with power cords, AWS rents us virtual computing capacity in professional data centers worldwide.

In our project, we use 5 specific AWS services in region ap-south-2 (Hyderabad):

- **Amazon S3 (Simple Storage Service):** A cloud storage service for files. We use S3 Static Website Hosting to hold our frontend HTML, CSS, and JavaScript files so anyone can load the website.
- **Amazon API Gateway (HTTP API v2):** A managed front-door router. It provides a public HTTPS web address (https://sqh1tjb8oj.execute-api.ap-south-2.amazonaws.com), accepts incoming browser requests, manages CORS security, and hands requests to Lambda.
- **AWS Lambda:** A serverless compute service. It runs our Node.js 22 backend code only when a request arrives. There is no server running 24/7; Lambda spins up in milliseconds, runs our code, and shuts down.
- **Amazon DynamoDB:** A high-speed, managed NoSQL database. It stores our endpoints, test suites, and test run history in permanent cloud tables that scale automatically.
- **Amazon CloudWatch:** A cloud monitoring and logging service. Whenever Lambda executes or hits an error, console.log statements are automatically streamed to CloudWatch (/aws/lambda/WebhookGuard-Backend) for live debugging.

8. HOW AWS WORKS IN THIS PROJECT (CLOUD ARCHITECTURE)

How the components fit together, and why your laptop does not need to stay on.

Here is the architecture flow diagram in simple text:

Browser (User on Phone or Laptop)

■ 1. Loads HTML/JS/CSS assets

▼

Amazon S3 Static Website Bucket (Frontend)

■ 2. User clicks 'Run Test' -> Sends API Request

▼

Amazon API Gateway (HTTP API v2 - Front Door)

■ 3. Triggers Lambda function

▼

AWS Lambda Function (WebhookGuard-Backend Engine)

■ 4. Outbound HTTP requests over the Internet

■■

▼ ▼

Target Webhook Endpoint Amazon DynamoDB

(e.g. Stripe, Custom API) (Stores Endpoints, Runs, Results)

■▲

■■ 5. Returns Response ■■■■■■■■ 6. Lambda writes results to DB

- **Why your laptop can be turned off:** All code and data reside in Amazon's Hyderabad data center. When someone opens the public URL, their phone or laptop connects directly to S3 and API Gateway. Your local machine is completely uninvolved.

9. WHAT IS CI/CD & HOW WEBHOOKGUARD SUPPORTS IT?

Explaining Continuous Integration, Continuous Delivery, and what is ACTUALLY implemented.

CI stands for Continuous Integration: Whenever a developer changes code, automated tests run immediately to ensure nothing broke.

CD stands for Continuous Delivery / Deployment: If the tests pass, the new code is automatically packaged and deployed to servers without human intervention.

Why CI/CD matters: It prevents bugs from reaching production customers and allows teams to release software updates multiple times a day safely.

- What is ACTUALLY implemented in WebhookGuard: We built a dedicated CI/CD Quality Gate API endpoint: POST /api/test-runs?failOnErrors=true. When called, Lambda executes the suite. If all tests pass, it returns HTTP 200 OK. If ANY test fails, it returns HTTP 422 Unprocessable Entity with a list of failures.
- How developers use it in GitHub Actions: A team can add one line to their GitHub Actions file (.github/workflows/deploy.yml) that sends a curl command to WebhookGuard. If WebhookGuard returns 422, the GitHub Action automatically stops the build and prevents bad code from going live.
- **What is NOT yet implemented:** WebhookGuard itself does not host a built-in CI/CD scheduler or code repository runner. It integrates as an external quality gate that existing CI/CD tools call.

10. WHAT IS HMAC CRYPTOGRAPHIC SECURITY?

How webhooks prove they are genuine without passwords, using the wax seal analogy.

HMAC stands for Hash-based Message Authentication Code. It uses cryptographic hashing (SHA-256) plus a shared secret key.

The Wax Seal Analogy: In ancient times, a king sent letters stamped with his royal ring into hot wax. The receiver didn't need to see the king in person; if the wax seal matched the royal crest and wasn't broken, they knew the letter was genuine and untampered.

- **Why it matters:** Webhook URLs are public on the internet. Anyone could send a fake request: { "event": "payment.success", "amount": 10000 }. Without a signature, your server might credit $10,000 to an attacker.
- **How it works:** The sender hashes the payload using a secret key only the sender and receiver know: Signature = HMAC_SHA256(payload, secret). The sender attaches this signature to the request header (e.g. X-Signature: sha256=...). The receiver recalculates the hash. If the hashes match, the message is authentic.
- **How WebhookGuard tests it:** WebhookGuard has an Invalid HMAC test. It intentionally alters one letter in the signature. If the target server accepts the bad signature with HTTP 200, WebhookGuard raises a SECURITY FAILURE alert!

12. TEST TYPES SUPPORTED IN WEBHOOKGUARD

The specific testing modules implemented in the backend TestRunnerService.

WebhookGuard supports 8 distinct test scenario types, covering positive, negative, performance, and security testing:

- **1. Valid Payload Test:** Sends a perfectly formatted JSON body with a valid HMAC signature. Tests the happy path. Expected: HTTP 200 OK.
- **2. Missing Required Field Test:** Intentionally removes mandatory keys (like transaction_id or amount). Tests if the API validates inputs. Expected: HTTP 400 Bad Request.

- **3. Malformed JSON Test:** Sends syntactically broken JSON with unclosed braces or invalid syntax. Tests if the server crashes (500) or handles parser errors gracefully. Expected: HTTP 400 Bad Request.
- **4. Empty Payload Body Test:** Sends a POST request with an empty body. Verifies the server handles missing payloads safely. Expected: HTTP 400 Bad Request.
- **5. Invalid Data Type Test:** Sends a string for a field that must be numeric (e.g. amount: "fifty"). Verifies type validation. Expected: HTTP 400 Bad Request.
- **6. Invalid HMAC Signature Test:** Calculates a corrupt signature. Tests whether forged requests from attackers are blocked. Expected: HTTP 401 Unauthorized.
- 7. Missing Authentication Header Test: Sends a request with no signature or API key at all. Verifies that public access without credentials is impossible. Expected: HTTP 401 Unauthorized.
- **8. Timeout SLA Test:** Sets a strict response threshold (e.g. 2000ms). Uses AbortController to verify if the server responds in time before SLA violation.
- **9. Retry & Recovery Test:** Evaluates how the engine retries requests (up to 3 attempts with exponential delay) when encountering transient HTTP 503 errors.

13. TEST SUITE, TEST RUN, TEST RESULT, & REPORT

Clearing up terminology with a simple school test analogy.

These four terms represent the lifecycle of testing in WebhookGuard:

- **Test Suite:** The question paper or blueprint. It is the saved template containing a list of test scenarios (e.g. 'Core Payment Reliability Suite' containing 10 questions).
- **Test Run:** The exam session. Every time you click 'Run Test Suite', a new Test Run record is created with a unique run ID, started timestamp, and completed timestamp.
- **Test Result:** The individual grade for each question. If a suite has 10 test scenarios, one Test Run creates 10 Test Result items, each recording actual status code, latency, and PASS/FAIL.
- **Report:** The final report card. It aggregates all test results into an executive summary showing overall pass rate (e.g. 80%), average latency (e.g. 111ms), and security warnings.

14. DATABASE EXPLANATION & DYNAMODB SCHEMA

How data is organized and stored permanently in Amazon DynamoDB.

In local development, WebhookGuard can use SQLite. In AWS cloud production, it uses Amazon DynamoDB across 4 distinct tables:

DynamoDB Table Name	Primary Key (Hash)	Secondary Index (GSI)	What Data is Stored
WebhookGuard-Endpoints	id (String UUID)	None	Webhook name, target URL, auth type, secret token, custom headers, environment
WebhookGuard-TestSuites	id (String UUID)	EndpointIndex (endpointId)	Suite name, description, target endpointId, array of test scenario definitions
WebhookGuard-TestRuns	id (String UUID)	EndpointIndex (endpointId, startedAt)	Test run ID, suite ID, trigger source (manual/api/cicd), status, total, passed, failed

WebhookGuard-TestResults	id (String UUID)	RunIndex (runId, timestamp)	Individual test outcome, test name, actual status, expected status, latency (ms), request/response body

15. WHY AWS LAMBDA FOR THIS PROJECT?

Honest engineering reasons for using serverless compute.

We chose AWS Lambda because it fits the exact traffic pattern of a webhook testing tool:

- **No Idle Server Cost:** Webhook tests run only when a developer clicks 'Run' or a CI/CD build triggers it. A traditional EC2 server costs money every second it sits doing nothing. Lambda costs $0 when idle.
- **Zero Server Maintenance:** No operating system updates, security patches, or Linux administration required.
- **Direct Outbound Internet Access:** Lambda executes outside a private VPC, meaning it can directly make high-speed outbound HTTP/HTTPS calls to any webhook URL in the world.
- **Automatic Scaling:** If 10 developers trigger test suites simultaneously, Lambda instantly spins up 10 isolated execution environments in parallel.
- **Fast Execution:** Our bundled Lambda package is only 547 KB, enabling cold starts in under 300 milliseconds.

16. WHY DYNAMODB INSTEAD OF SQLITE IN AWS?

Why local file databases fail in serverless architectures.

In local development on a laptop, SQLite is great because it is a single file on your hard drive. But in AWS Lambda, SQLite fails for two critical reasons:

- **Lambda File System is Ephemeral:** Lambda instances are destroyed and recreated continuously. Any SQLite file written inside Lambda would be wiped out as soon as the instance shuts down.
- **Multi-Instance Concurrency:** If two Lambda functions run at the same time, they cannot safely share a local SQLite file without locking conflicts or data corruption.
- **Why DynamoDB solves this:** DynamoDB is a fully managed cloud database. Any Lambda instance anywhere can read and write data in single-digit milliseconds, and data persists permanently.

17. WHY API GATEWAY INSTEAD OF DIRECT LAMBDA CALLS?

Why browsers cannot directly invoke Lambda functions.

A browser cannot directly speak to a raw Lambda function. API Gateway acts as the bridge:

- **Public HTTPS Address:** API Gateway provides the public URL that browsers and curl scripts can call over the internet.
- **HTTP Protocol Translation:** It translates standard HTTP requests (headers, query params, body) into the JSON event format that Lambda expects.
- **CORS Management:** It handles Cross-Origin Resource Sharing (CORS) so that our S3-hosted frontend website can make API calls to the gateway without browser security blocks.
- **Security & Rate Limiting:** It protects Lambda from denial-of-service traffic and throttles abusive requests.

18. CLOUDWATCH: LOGGING & MONITORING

How we see what is happening inside our serverless backend.

Because Lambda has no monitor or physical screen, we rely on Amazon CloudWatch Logs:

- **Automatic Log Ingestion:** Every console.log() and console.error() in our Node.js code is automatically captured and streamed to the log group /aws/lambda/WebhookGuard-Backend.
- **Performance Telemetry:** CloudWatch logs every invocation with exact duration (e.g. Duration: 211ms), billed duration, and memory used (e.g. 89 MB out of 512 MB).
- **Debugging:** During deployment, when an esbuild bundling error occurred, we ran aws logs filter-log-events via AWS CLI, saw the exact error line in 2 seconds, and fixed it immediately.

19. HOW THE PROJECT WAS DEPLOYED TO AWS

The deployment pipeline used to get WebhookGuard live.

The deployment was executed autonomously using the AWS CLI and CloudFormation:

- **Step 1 (Packaging):** The TypeScript backend was bundled using esbuild into a single self-contained Node.js file (dist-lambda/index.js, 547 KB compressed) with all AWS SDK and Express dependencies included.
- **Step 2 (Staging):** The zip file was uploaded to an S3 deployment bucket (s3://webhookguard-deploy-792207721676-ap-south-2/lambda.zip).
- **Step 3 (Infrastructure as Code):** We defined the entire cloud stack in template.yaml and deployed it with aws cloudformation deploy. CloudFormation provisioned the 4 DynamoDB tables, IAM execution role, Lambda function, API Gateway, and S3 website bucket.
- **Step 4 (Frontend Build & Sync):** We built the React frontend with the live API Gateway URL in .env.production and synced the assets to the S3 website bucket using aws s3 sync.
- **Step 5 (Database Seeding):** We executed a POST /api/seed request to pre-populate DynamoDB with realistic demo webhooks and test suites.

20. AWS CLI COMMANDS EXPLAINED

Key command-line operations used during the setup.

The AWS CLI (Command Line Interface) is a terminal tool that lets engineers manage AWS cloud resources with commands instead of clicking through the web console.

- **aws sts get-caller-identity:** Checks who is logged in. Returns your AWS Account ID, IAM User, and ARN to verify credentials work.
- **aws cloudformation deploy:** Reads a template.yaml file and automatically creates or updates all the cloud resources defined in it.
- **aws s3 sync:** Copies a local directory (like our frontend build) to an Amazon S3 bucket, uploading only modified files.
- **aws lambda update-function-code:** Uploads a new zip file directly to an existing Lambda function without tearing down infrastructure.
- **aws dynamodb scan:** Reads items directly from a DynamoDB table to verify that records were saved successfully.

21. DEMO SCRIPT: HOW TO PRESENT WEBHOOKGUARD (STEP-BY-STEP)

A 2-minute demonstration flow to impress your examiner or interviewer.

Follow these exact steps when demonstrating WebhookGuard:

- **1. Open Website:** Open http://webhookguard-frontend-792207721676-ap-south-2.s3-website.ap-south-2.amazonaws.com in Chrome or Edge.
- **2. Show Dashboard:** Point out the KPI cards (Total Tests, Passed, Failed, Success Rate, Average Latency) and explain that these stats are fetched live from AWS DynamoDB.
- **3. Show Endpoints:** Click 'Endpoints'. Show the registered targets (e.g. Stripe / Adyen Payment Webhook) and point out that secrets are masked for security.

- **4. Show Test Suites:** Click 'Test Suites'. Open 'Core Payment Reliability & Security Suite' and show the variety of scenarios: valid payload, missing transaction ID, malformed JSON, invalid HMAC, timeout, and retry.
- **5. Click 'Run Test Suite':** Trigger the test run. Point out the live progress bar and watch the status badges update.
- **6. Inspect Test Results:** Click on 'Invalid HMAC Signature' -> show the 401 Unauthorized response. Click on 'Malformed JSON' -> show the 400 response.
- **7. Point out Security Failure:** Show the 'Missing Authentication Header' scenario where the vulnerable target returned 200 OK, and explain how WebhookGuard flagged it as a SECURITY FAILURE.
- **8. Refresh Page:** Refresh the browser and show that the test run and report are still there, proving cloud persistence in DynamoDB.
- **9. Conclude:** 'The entire application is serverless on AWS, requires zero local servers, and costs pennies to run.'

25. 30-SECOND, 1-MINUTE, & 3-MINUTE VERBAL PITCHES

Memorize the version that matches the time your interviewer gives you.

--- Version A: The 30-Second Elevator Pitch ---

"I built WebhookGuard, an automated testing and security platform for webhook APIs. When services like Stripe or GitHub notify your backend, bad payloads or fake signatures can cause downtime or financial fraud. WebhookGuard lets developers test their webhooks against broken JSON, timeouts, and forged HMAC signatures before deploying. It is fully serverless on AWS using S3, API Gateway, Lambda, and DynamoDB, running live 24/7 without needing my computer on."

--- Version B: The 1-Minute Comprehensive Pitch ---

"My project is called WebhookGuard. It solves a critical problem in modern API architecture: webhook reliability. Webhooks are event-driven HTTP notifications sent between systems, like Stripe notifying an e-commerce store of a payment. But if an endpoint cannot handle missing fields, malformed JSON, high latency, or forged signatures, serious bugs happen.

WebhookGuard provides a full-stack platform where developers register endpoints and run pre-configured test suites. Our AWS Lambda backend sends real HTTP requests over the internet, records millisecond response times, and flags security vulnerabilities. The results are saved to Amazon DynamoDB and visualized on a React dashboard hosted on S3. It also includes a CI/CD endpoint returning HTTP 422 to automatically block bad deployments in GitHub Actions."

--- Version C: The 3-Minute In-Depth Pitch ---

"I'd love to walk you through WebhookGuard. WebhookGuard is an automated testing and reliability engine designed specifically for webhook receiver APIs.

The Motivation: In traditional web development, testing outgoing APIs is common, but testing incoming webhooks is often neglected. If a payment webhook crashes on malformed data or accepts an invalid signature, the consequences range from lost revenue to unauthorized account access.

The Architecture: I built this as a cloud-native serverless system on AWS. The frontend is a React and TypeScript Single Page App styled with Tailwind CSS, hosted on an Amazon S3 static website bucket. The API layer uses Amazon API Gateway HTTP API v2, which routes requests to an AWS Lambda function running Node.js 22. For storage, I implemented a repository pattern that uses SQLite for local offline development, but automatically connects to Amazon DynamoDB in the cloud across four on-demand tables.

The Engine: When a test runs, Lambda executes real outbound HTTP requests. It tests positive payloads, negative cases like missing fields and syntax errors, timeout SLA limits using AbortController, and retry recovery. Crucially, it tests cryptographic security by intentionally sending bad HMAC-SHA256 signatures. If a server incorrectly accepts a bad signature, WebhookGuard flags a Security Failure.

Everything is deployed live in AWS region ap-south-2 Hyderabad. It requires zero server maintenance, scales automatically, and integrates with CI/CD quality gates."

11. HTTP STATUS CODES IN WEBHOOKGUARD

Code	Name	Plain English Meaning	How WebhookGuard Tests It
200	**OK**	The server understood and processed the request successfully.	Expected for valid payloads, idempotency replays, and successful retry attempts.
201	**Created**	The request succeeded and a new resource was created.	Returned when an endpoint creates a new webhook delivery record.
204	**No Content**	Request succeeded, but there is no message body to return.	Returned by API Gateway for CORS OPTIONS preflight checks.
400	**Bad Request**	The client sent invalid data (e.g. malformed JSON or missing fields).	Expected for negative tests (missing transaction_id, broken JSON, wrong data types).
401	**Unauthorized**	Request lacked valid authentication or had a bad signature.	Expected when WebhookGuard tests invalid HMAC signatures or missing auth headers.
403	**Forbidden**	The server understood the credentials, but refuses access.	Returned when an IP address or API key is explicitly barred.
404	**Not Found**	The requested URL or resource does not exist on the server.	Returned when an invalid endpoint ID or test run ID is requested.
422	**Unprocessable Entity**	Request syntax was valid, but contained semantic test failures.	Returned by WebhookGuard's CI/CD endpoint (?failOnErrors=true) to fail build pipelines.
429	**Too Many Requests**	The client exceeded the allowable rate limit.	Expected when testing burst traffic and rate limiting resilience.
500	**Internal Server Error**	The server crashed due to an unhandled exception.	Considered a test failure when an endpoint crashes on malformed input.
502	**Bad Gateway**	A gateway or proxy received an invalid response upstream.	Encountered when a proxy or serverless container crashes during execution.
503	**Service Unavailable**	The server is temporarily overloaded or down for maintenance.	Used in retry testing to verify that WebhookGuard attempts exponential backoff.

22. COMMON QUESTIONS I MAY BE ASKED (VIVA / INTERVIEW Q&A;)

Q1: What is WebhookGuard?

- WebhookGuard is an automated testing platform for webhook APIs that validates payload structure, error handling, latency, and cryptographic HMAC security.

Q2: Why did you choose this project?

- Because webhooks are mission-critical in modern systems (payments, orders, CI/CD), but developers rarely have automated tools to stress-test and security-audit their webhook endpoints before production.

Q3: What problem does it solve?

- It prevents webhook outages, crashes from malformed JSON, silent failures, security vulnerabilities from forged signatures, and slow response timeouts.

Q4: What is a webhook?

- A webhook is an automated HTTP POST request sent by a source system (like Stripe or GitHub) to a target API when a specific event occurs.

Q5: What is an API?

- An Application Programming Interface (API) is a structured set of rules and protocols that allows two software programs to communicate over a network.

Q6: What is REST?

- Representational State Transfer (REST) is an architectural style for web APIs that uses standard HTTP methods (GET, POST, PUT, DELETE) and JSON data.

Q7: What is JSON?

- JavaScript Object Notation (JSON) is a lightweight, human-readable text format for storing and exchanging structured key-value data.

Q8: What is HTTP?

- Hypertext Transfer Protocol (HTTP) is the foundational request-response communication protocol used to transmit web pages and API data over the internet.

Q9: What is AWS Lambda?

- AWS Lambda is a serverless compute service that runs backend code in response to events or HTTP requests without provisioning or managing virtual servers.

Q10: Why did you choose Lambda for the backend?

- Because webhook testing is bursty and on-demand. Lambda costs $0 when idle, scales automatically, requires no OS management, and directly executes outbound HTTP calls.

Q11: What is Amazon API Gateway?

- API Gateway is a managed AWS service that provides public HTTPS URLs, handles CORS headers, throttles requests, and forwards incoming HTTP traffic to Lambda.

Q12: Why use API Gateway instead of exposing Lambda directly?

- Browsers cannot directly call raw Lambda functions easily. API Gateway provides the public HTTPS interface, manages CORS security, and translates web requests for Lambda.

Q13: What is Amazon DynamoDB?

- DynamoDB is a fully managed, serverless NoSQL database that provides fast, single-digit millisecond response times and automatically scales on demand.

Q14: Why use DynamoDB instead of a traditional SQL database?

- DynamoDB is completely serverless (no idle server costs), scales automatically to zero, and has native pay-per-request pricing that matches Lambda perfectly.

Q15: What is Amazon S3?

- S3 (Simple Storage Service) is cloud object storage. We use its Static Website Hosting feature to host our compiled React, HTML, CSS, and JS files.

Q16: What is Amazon CloudWatch?

- CloudWatch is AWS's monitoring and logging tool. All console.log statements and error traces from Lambda are automatically recorded there for debugging.

Q17: What is CI/CD?

- Continuous Integration and Continuous Deployment (CI/CD) is an automated software practice where code changes are automatically built, tested, and deployed to production.

Q18: Is CI/CD implemented in your project?

- Yes, as a Quality Gate API! We implemented POST /api/test-runs?failOnErrors=true, which returns HTTP 422 if any test fails so external CI/CD tools (like GitHub Actions) can block bad releases.

Q19: What is HMAC?

- HMAC (Hash-based Message Authentication Code) is a cryptographic algorithm that hashes a payload using a secret key, ensuring the message came from a trusted sender and wasn't modified.

Q20: Why is HMAC important for webhooks?

- Because webhook URLs are publicly exposed on the internet. HMAC signatures prevent attackers from sending forged requests (e.g. fake payment success notifications).

Q21: How does your system test an API?

- Lambda dispatches real outbound HTTP requests with custom headers, payloads, and signatures to the target URL, measuring response time and matching returned vs expected status codes.

Q22: How do you handle errors during testing?

- We wrap requests in try-catch blocks, catch network failures and DNS errors, record the error message in the TestResult object, and return a clean JSON summary without crashing.

Q23: How do you handle timeouts?

- We use JavaScript's native AbortController with a configured timeout in milliseconds. If the target server takes too long, the controller aborts the request and marks it as a timeout failure.

Q24: Where are test results stored?

- In AWS production, they are stored in the Amazon DynamoDB table WebhookGuard-TestResults. In local development, they are stored in a local SQLite file.

Q25: Does my laptop need to stay on after deployment?

- No! Everything runs on AWS in the cloud (S3, API Gateway, Lambda, DynamoDB). The website and backend work 24/7 even when your laptop is turned off.

Q26: What happens if the target webhook is unavailable?

- WebhookGuard catches the connection refusal or 503 error, records the failure, executes configured retry attempts, and logs the details in the test report.

Q27: What are the limitations of the current implementation?

- Lambda has a maximum execution limit of 30 seconds (configured in our template), so single test suites cannot run for more than 30 seconds without chunking or async queues.

Q28: What would you improve in the future?

- I would add scheduled recurring cron tests using AWS EventBridge, webhook failure alerts via email/SMS using Amazon SNS, and support for asynchronous SQS message queuing.

23. QUESTIONS THAT MAY BE ASKED TO TEST MY UNDERSTANDING (DEEP DIVE)

Deep Dive Q1: Why not use a traditional server (like EC2 or a VPS) instead of serverless?

- A traditional EC2 server requires 24/7 billing, OS patching, security hardening, and manual scaling. Webhook testing is bursty: hours of inactivity followed by a test run. Serverless scales to zero cost when idle and requires zero server maintenance.

Deep Dive Q2: Why couldn't you keep using SQLite in AWS Lambda?

- AWS Lambda is stateless and ephemeral. The container filesystem is wiped out when the instance spins down. Furthermore, if two Lambda containers run simultaneously, they cannot share a local SQLite file without corruption. DynamoDB provides persistent, concurrent cloud storage.

Deep Dive Q3: What happens if a target webhook takes 40 seconds to respond?

- Our Lambda function has a configured timeout of 30 seconds. Furthermore, WebhookGuard uses AbortController with an endpoint-level SLA timeout (e.g. 5000ms). The request is aborted and marked as a timeout failure long before the Lambda container timeout is reached.

Deep Dive Q4: How does retry logic work in your test runner?

- When a test scenario specifies retryCount (e.g. 3 attempts) and retryDelayMs (e.g. 400ms), the runner executes the initial request. If it fails with an unexpected status (like 503), it pauses for the delay and retries up to the maximum count, recording each attempt's status and latency.

Deep Dive Q5: How does the frontend know the API Gateway URL in production?

- During the frontend build process, Vite reads the environment variable VITE_API_URL from .env.production, which contains our API Gateway endpoint. Vite injects this URL directly into the compiled JavaScript bundle.

Deep Dive Q6: What are the limitations of a serverless architecture?

- Serverless architectures can experience 'cold starts' (a few hundred milliseconds delay on the first request after idle), hard execution time limits (max 15 minutes in Lambda, 30 seconds configured in our API Gateway), and statelessness requiring external databases like DynamoDB.

Deep Dive Q7: How does WebhookGuard handle CORS across different domains?

- The frontend is hosted on S3 (s3-website domain) while the backend is on API Gateway (execute-api domain). API Gateway is configured in template.yaml with CorsConfiguration (AllowOrigins: '*', AllowMethods, AllowHeaders), returning HTTP 204 No Content for OPTIONS preflights.

24. SIMPLE DEFINITIONS CHEAT SHEET

Term	Simple Meaning (Plain English)	Real WebhookGuard Example
API	Application Programming Interface — rules for two programs to talk.	/api/endpoints, /api/test-runs
Webhook	An automated HTTP POST notification sent when an event occurs.	Stripe notifying store when payment succeeds
HTTP	Hypertext Transfer Protocol — web communication standard.	HTTP/1.1 and HTTP/2 requests
HTTPS	Encrypted HTTP with SSL/TLS security.	https://sqh1tjb8oj.execute-api...
REST	Architectural style using standard HTTP verbs and JSON.	GET, POST, PUT, DELETE routes in Express
JSON	JavaScript Object Notation — text format for data.	{"event": "payment.success", "amount": 4999}
Request	The message sent by the client asking for action.	POST request sent to the webhook receiver
Response	The message returned by the server with status and data.	HTTP 200 OK with {"success": true}
Header	Metadata sent alongside request/response bodies.	Content-Type: application/json, X-Signature
Payload	The actual data body inside the HTTP request.	The JSON body containing transaction details
Endpoint	A specific URL where an API or webhook receives requests.	https://api.shoestore.com/webhooks/billing
Status Code	3-digit number indicating the result of an HTTP request.	200 (Success), 400 (Bad Data), 401 (Unauthorized)
Frontend	The user interface that runs in the browser.	React + TypeScript + Vite + Tailwind UI
Backend	The server code that processes logic and talks to DB.	Node.js 22 + Express running inside Lambda
Serverless	Cloud computing where cloud provider manages servers.	AWS Lambda + DynamoDB (no virtual machines)
AWS	Amazon Web Services — largest cloud platform.	Our cloud host in region ap-south-2 (Hyderabad)
Lambda	AWS serverless function compute service.	WebhookGuard-Backend function (Node.js 22)
API Gateway	AWS front-door routing service for APIs.	WebhookGuard-Api (HTTP API v2)
DynamoDB	AWS serverless NoSQL key-value database.	WebhookGuard-Endpoints, WebhookGuard-TestRuns
S3	AWS Simple Storage Service for file hosting.	webhookguard-frontend-792207721676-ap-south-2
CloudWatch	AWS logging and monitoring dashboard.	/aws/lambda/WebhookGuard-Backend log group

IAM	Identity and Access Management — permissions in AWS.	WebhookGuard-LambdaExecutionRole
AWS CLI	Command Line Interface to control AWS from terminal.	aws cloudformation deploy, aws s3 sync
CloudFormation	AWS Infrastructure as Code service.	template.yaml defining all cloud resources
HMAC	Hash-based Message Authentication Code using secret key.	HMAC-SHA256 signature on webhook payloads
SHA-256	Secure cryptographic 256-bit hashing algorithm.	Generates 64-character hex signature string
CI/CD	Continuous Integration / Continuous Deployment.	Automated building, testing, and shipping code
Quality Gate	A check that blocks deployment if tests fail.	POST /api/test-runs?failOnErrors=true (returns 422)
Timeout	Maximum time to wait before cancelling request.	AbortController enforcing 2000ms SLA limit
Retry	Attempting a failed request again after a delay.	Retrying transient 503 errors up to 3 times
Test Suite	A saved collection of test scenarios.	Core Payment Reliability & Security Suite
Test Run	A single execution instance of a test suite.	Run ID: ea8ec7ec-df54-4f73-9512-935e56ba1d06
Test Result	The granular outcome of one test scenario.	Valid Payment Payload -> PASS (688ms)